ASV Sim Technical Documentation

January 29, 2026

Abstract

This application presents the design, implementation, and mathematical validation of a control center for an Autonomous Surface Vessel (ASV). The system functions as a Digital Testing Environment (DTE), providing a high-fidelity interface for real-time monitoring, teleoperation, and autonomous trajectory planning. By integrating a First-Order Nomoto model with a Python-based graphical interface and a Robot Operating System 2 (ROS 2) communication backend, the system demonstrates robust capabilities in kinematic simulation and guidance law execution. This document details the system architecture, the derivation of the governing physical equations, and the operational protocols for the end-user.

Contents

1 Introduction **3**
1.1 Project Purpose . . . 3
1.2 Scope and Limitations . . . 3
1.2.1 Operational Envelope . . . 3
1.2.2 Control and Guidance Capabilities . . . 4

2 System Overview **5**
2.1 System Architecture . . . 5
2.1.1 Data Flow and Node Structure . . . 5
2.2 Coordinate Systems and Transformations . . . 5
2.2.1 Reference Frames . . . 6
2.2.2 Rotational Transformations (Quaternion to Euler) . . . 6
2.3 Mathematical Model Derivation . . . 8
2.3.1 Steering Dynamics (Yaw) . . . 8
2.3.2 Surge Dynamics (Speed) . . . 8
2.4 Sensor Systems and Signal Processing . . . 9
2.4.1 Communication Architecture . . . 9
2.4.2 Global Navigation Satellite System (GNSS) . . . 9
2.4.3 Inertial Measurement Unit (IMU) . . . 11
2.4.4 Rudder Actuation Feedback . . . 12
2.4.5 Data Synchronization and Fusion . . . 13
2.5 Control System Design . . . 14
2.5.1 Control Architecture Overview . . . 14
2.5.2 Manual Control Modes . . . 15
2.5.3 Automatic Control (PID) . . . 16
2.6 Trajectory and Mission Planning . . . 16
2.6.1 Path Planner . . . 17
2.6.2 Rudder Sequence Scheduler . . . 18
2.7 Software Initialization . . . 18
2.8 Main Dashboard Overview . . . 19
2.9 Simulation Lifecycle Management . . . 20
2.10 Control Modes . . . 21
2.10.1 Manual Control (Sliders & Keyboard) . . . 21
2.11 Trajectory and Mission Planning . . . 22
2.11.1 Waypoint Navigation . . . 22
2.11.2 Rudder Sequence Scheduler . . . 23
2.12 Parameter Tuning . . . 24
2.13 Real-Time Visualization . . . 25

2.13.1 Time-Series Plots 25
2.13.2 2D Trajectory Map 26
2.14 Data Logging and Export 27
2.14.1 Visual Trace Management 27
2.14.2 Telemetry Export 27

Chapter 1

Introduction

1.1 Project Purpose

The primary objective of this project is to establish a high-fidelity, cost-effective Digital Testing Environment (DTE) designed for the rapid prototyping and validation of marine control systems. In the domain of maritime robotics, the transition from theoretical design to open-water deployment is fraught with logistical complexity, high costs, and operational risks. This project addresses these challenges by providing a realistic *Software-in-the-Loop (SiL)* testing architecture.

Specifically, the system serves to:

- **Facilitate Modular Integration:** The architecture allows researchers to integrate and test novel Guidance, Navigation, and Control (GNC) solutions within an environment that mimics realistic sensor noise and data rates.
- **Enable Universal Vessel Modeling:** Through the implementation of updatable parameters, the system can be dynamically reconfigured to represent a diverse range of marine vessel behavior, from small catamarans to larger displacement hulls, without code recompilation.
- **Real-Time Validation:** The system guarantees real-time kinematic solving, ensuring that control loops are tested under strict timing constraints analogous to physical hardware deployment.

1.2 Scope and Limitations

The current version of the ASV Sim is defined by the following operational boundaries and functional capabilities:

1.2.1 Operational Envelope

- **Simulation Paradigm:** The system operates strictly as a Software-in-the-Loop (SiL) platform. Hardware-in-the-Loop (HiL) integration is currently outside the project boundaries and will be implemented in the later versions.
- **Environmental Conditions:** The physics engine assumes a "calm water" environment. Second-order wave drift forces, wind load, and ocean current disturbances are not modeled in this version.

- **Physics Fidelity:** The vessel dynamics are governed by a 3-DOF (Surge, Sway, Yaw) maneuvering model suitable for steering and speed control.

1.2.2 Control and Guidance Capabilities

- **Actuation Logic:** Control is achieved via distinct PID (Proportional-Integral-Derivative) loops governing Surge Speed and Heading.
- **Trajectory Management:** The system employs a Waypoint-based Line-of-Sight (LOS) guidance algorithm. Advanced trajectory tracking methods (e.g., Dubins paths or continuous curvature smoothing) and dynamic obstacle-aware path planning strategies are excluded from the current scope and are reserved for future development.
- **Sensor Interfaces:** The system simulates a multi-modal navigational suite compliant with standard Robot Operating System 2 (ROS 2) message definitions. Telemetry streams include Odometry (Position/Velocity) and Joint States (Rudder Feedback), augmented by virtualized Global Navigation Satellite System (GNSS) and Inertial Measurement Unit (IMU) interfaces to support the development and validation of robust sensor fusion algorithms.

Chapter 2

System Overview

This chapter details the structural and mathematical framework of the ASV Control Center. The system is architected as a modular DTE, where a high-fidelity physics kernel interacts with a ROS 2 communication layer to simulate the kinematic and dynamic response of a surface vessel in real-time.

2.1 System Architecture

The software follows a decoupled publisher-subscriber pattern utilizing the ROS 2 middleware. The core logic is encapsulated within the `GUI Node`, which functions as both the simulation solver and the interface bridge.

2.1.1 Data Flow and Node Structure

The architecture is defined by three primary data loops operating at a synchronous frequency of 20Hz ($T_s = 50ms$):

1. **State Estimation Loop:** Subscribes to the navigational telemetry topics (odometry, GPS, IMU) and actuator feedback to reconstruct the full 6-DOF state vector of the vessel.

2. **Physics Loop:** Solves the differential equations of motion based on the computed control efforts and current hydrodynamic parameters to update the simulation state for the next time step.

3. **Control Loop:** Processes user inputs (Manual or Autonomous Setpoints) and computes the required control efforts (Thrust and Rudder Angle) using the active PID control laws.

2.2 Coordinate Systems and Transformations

To accurately simulate navigation, the system employs two distinct reference frames in accordance with SNAME (1950) notation.

2.2.1 Reference Frames

- **North-East-Down (NED):** The inertial earth-fixed frame, denoted as $\{n\} = (x_n, y_n, z_n)$. Here, x_n points true North, y_n points East, and z_n points Down.
- **Body-Fixed Frame:** The moving frame attached to the vessel, denoted as $\{b\} = (u, v, r)$. The origin is located at the center of gravity (CG). u represents surge velocity, and r represents yaw rate.

2.2.2 Rotational Transformations (Quaternion to Euler)

The state estimation module processes orientation data provided by the ROS 2 odometry stream. While the system's control logic operates on intuitively readable Euler angles (specifically the Yaw heading), the underlying communication layer utilizes Unit Quaternions. This design choice is critical for avoiding mathematical singularities known as "Gimbal Lock" during transmission.

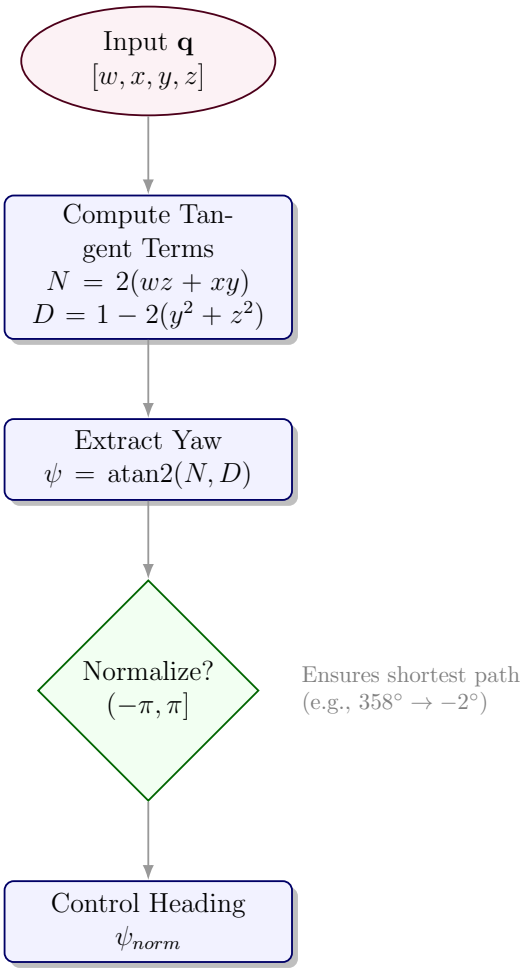

Figure 2.1: Heading Extraction

Euler Angles and the Singularity Problem

Euler angles describe orientation as a sequence of three elemental rotations around body-fixed axes. For marine surface vessels, the standard Tait-Bryan ZYX convention is employed:

- **Yaw (ψ):** Rotation about the z-axis.
- **Pitch (θ):** Rotation about the y-axis.
- **Roll (ϕ):** Rotation about the x-axis.

However, Euler angle representations suffer from intrinsic singularities. If the Pitch angle θ approaches $\pm 90°$, the Roll and Yaw axes become aligned (collinear), reducing the degrees of freedom to two. This condition, termed Gimbal Lock, causes the mathematical solution for orientation to become undefined or unstable.

Unit Quaternions

To mitigate this, the system utilizes Unit Quaternions for robust state tracking. A quaternion $\mathbf{q}$ is a hyper-complex number system extending the complex plane into four dimensions:

$$\mathbf{q} = q_w + q_x\mathbf{i} + q_y\mathbf{j} + q_z\mathbf{k} \tag{2.1}$$

represented vectorially as $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$, subject to the unit norm constraint $||\mathbf{q}|| = 1$. This representation provides a continuous, singularity-free mapping of the rotation group $SO(3)$.

Derivation of Heading (Yaw)

To extract the control-relevant Yaw angle ψ from the quaternion, the quaternion is first mapped to the Direction Cosine Matrix (Rotation Matrix) $R_b^n(\mathbf{q})$. The general form of this matrix in terms of quaternion elements is given by:

$$R_b^n(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \tag{2.2}$$

For a planar navigation problem where the primary variable of interest is the heading ψ, the relationship between the body-frame x-axis projected onto the inertial frame is examined. This corresponds to the matrix elements R_{11} and R_{21} (the first column of the rotation matrix), which relate to the tangent of the yaw angle:

$$\tan(\psi) = \frac{R_{21}}{R_{11}} = \frac{2(q_wq_z + q_xq_y)}{1 - 2(q_y^2 + q_z^2)} \tag{2.3}$$

By applying the four-quadrant inverse tangent function (atan2), the full heading angle is recovered in the range $(-\pi, \pi]$:

$$\psi = \text{atan2}\left(2(q_wq_z + q_xq_y), 1 - 2(q_y^2 + q_z^2)\right) \tag{2.4}$$

Domain Normalization

The raw calculated angle may exceed standard bounds during continuous rotation (e.g., multiple full turns). For the PID controller to function correctly without "unwinding" behavior, the error signal is normalized to the shortest-path domain:

$$\psi_{norm} = \text{atan2}(\sin(\psi), \cos(\psi)) \tag{2.5}$$

This ensures that a commanded turn from 179° to −179° is treated as a 2° right turn rather than a 358° left turn.

2.3 Mathematical Model Derivation

The vessel's maneuvering dynamics are approximated using a phenomenological "black-box" model, specifically the First-Order Nomoto Model. This model abstracts the complex hydrodynamic interactions (added mass, damping, Coriolis forces) into two identifiable parameters: indices K and T.

2.3.1 Steering Dynamics (Yaw)

The Nomoto model describes the transfer function between the rudder angle δ and the yaw rate r:

$$\frac{r(s)}{\delta(s)} = \frac{K}{1+Ts} \tag{2.6}$$

In the time domain, this is expressed as the differential equation:

$$T\dot{r} + r = K\delta \tag{2.7}$$

Where:

- K $[s^{-1}]$: The gain constant, representing the steady-state turning velocity per unit rudder angle.
- T $[s]$: The time constant, representing the vessel's inertial response lag.

Non-Linear Parameter Variation

To capture the non-linear hydrodynamic damping inherent in marine vessels, K and T are not static. The system implements a dynamic scheduling algorithm based on the rudder angle magnitude $|\delta|$:

$$K(\delta) = K_{base} + K_{\delta}|\delta| \tag{2.8}$$

$$T(\delta) = T_{base} + T_{\delta}|\delta| \tag{2.9}$$

This allows the simulation to replicate the loss of turning efficiency at high rudder angles (stall regions) or varying hull characteristics.

2.3.2 Surge Dynamics (Speed)

Longitudinal motion is modeled as a first-order system with non-linear drag coupling. The surge acceleration $\dot{u}$ is derived from the balance of commanded thrust and hydrodynamic resistance:

$$\dot{u} = \frac{1}{\tau_u}(u_{cmd} - u) - X_{added}(r) \tag{2.10}$$

Where X_{added} represents the added resistance due to maneuvering (speed bleed). Following standard steering equations (Norrbin, 1971), this is modeled as a non-linear damping term proportional to the square of the yaw rate:

$$X_{added}(r) = C_d \cdot r^2 \tag{2.11}$$

This term ensures that the vessel realistically decelerates during turns regardless of direction, mimicking the increased hydrodynamic cross-section presented to the flow.

2.4 Sensor Systems and Signal Processing

The ASV Control Center utilizes a closed-loop feedback architecture driven by a suite of virtualized environmental sensors. To facilitate Software-in-the-Loop (SiL) validation, the system acts as both the *generator* of sensor data (simulating physical hardware) and the *processor* of that data (simulating the control computer).

2.4.1 Communication Architecture

The sensor interface is abstracted through the ROS 2 middleware. The communication topology follows a synchronous Publisher-Subscriber pattern operating at a deterministic frequency of 20Hz ($T_s = 50ms$).

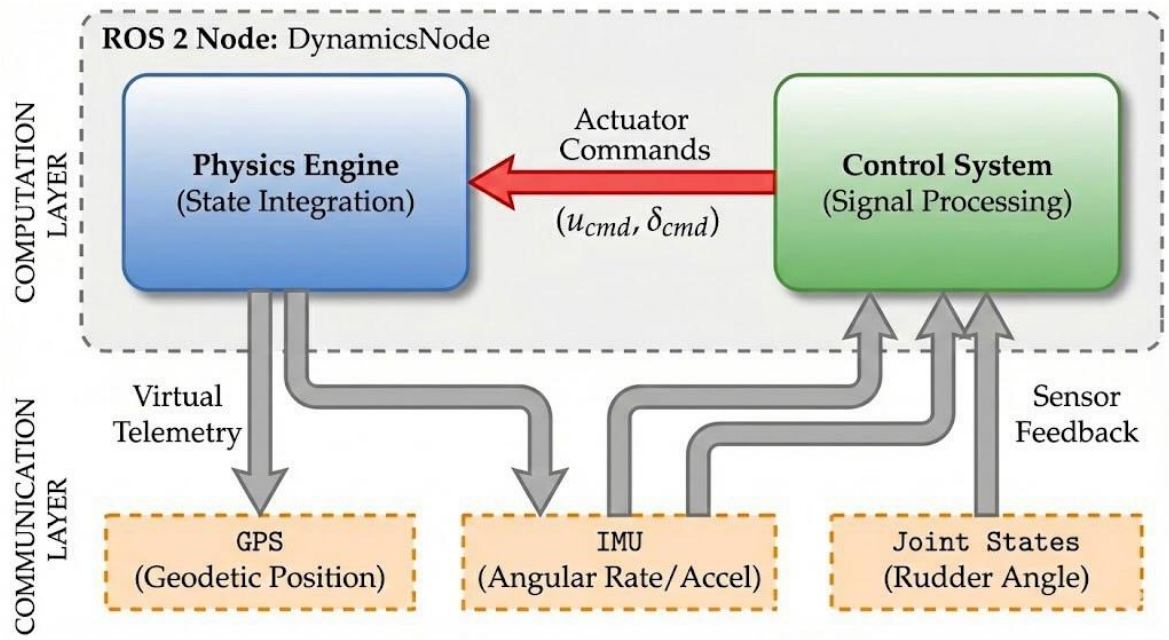

Figure 2.2: Communication Topology

As illustrated in Figure 2.2, the data flow is bidirectional:

1. **Physics Layer:** The `Dynamics Node` solves the differential equations of motion and injects the resulting state vector into standard ROS message types.
2. **Control Layer:** The navigation stack ingests these messages, applies signal conditioning, and computes the error signals for the PID controllers.

2.4.2 Global Navigation Satellite System (GNSS)

Position tracking is managed by a virtualized GNSS receiver mimicking the behavior of a standard GPS module.

Publisher Node

The simulation module converts the local Cartesian position (x, y) in meters into Geodetic coordinates (Latitude ϕ, Longitude λ). The system employs an Equirectangular Projection model, assuming a local flat-earth approximation where longitudinal convergence is negligible for the operational range ($< 5km$).

The transformation used by the publisher on the `gps` topic is:

$$\phi_{sim}(t) = \phi_{ref} + \left(\frac{y(t)}{R_{scale}}\right) \tag{2.12}$$

$$\lambda_{sim}(t) = \lambda_{ref} + \left(\frac{x(t)}{R_{scale}}\right) \tag{2.13}$$

In the provided implementation, the uniform scaling factor R_{scale} is linearized to a constant $111,139$ m/deg, representing the average meridional radius of curvature.

Receiver Node

The control node subscribes to the GNSS topic. To utilize this data for velocity control, the system must differentiate the positional stream to recover the velocity vector.

1. Coordinate Re-projection: The incoming Geodetic stream is projected back to the local Tangent Plane using the inverse of the simulation scaling:

$$x_{meas} = (\lambda_{meas} - \lambda_{ref}) \cdot 111139.0 \tag{2.14}$$

$$y_{meas} = (\phi_{meas} - \phi_{ref}) \cdot 111139.0 \tag{2.15}$$

2. Velocity Estimation (Finite Difference): Since standard consumer GPS units often output position at a lower rate than the control loop, the system derives the linear velocity vector $\mathbf{v}_{lin}$ using a First-Order Backwards Difference method:

$$\mathbf{v}_{lin}[k] = \frac{\mathbf{p}_{meas}[k] - \mathbf{p}_{meas}[k-1]}{\Delta t} \tag{2.16}$$

Where $\mathbf{p}_{meas} = [x_{meas}, y_{meas}]^T$. This derived velocity serves as the process variable (PV) for the Surge Speed PID controller.

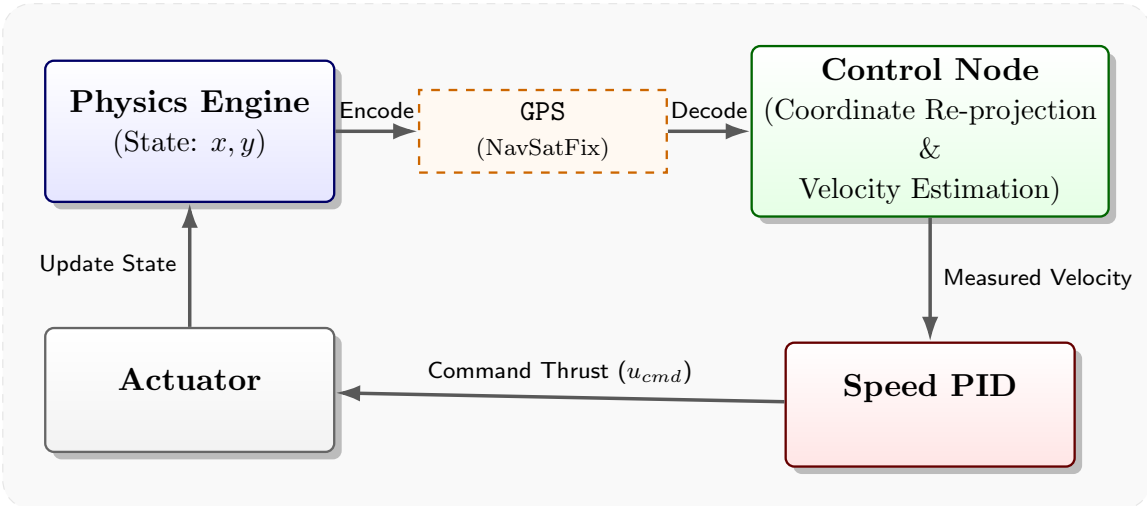

Figure 2.3: GNSS Control Topology: A structured view of the closed-loop architecture. The upper layer handles the sensor simulation pipeline (Physics → Encoding → Decoding → Velocity Estimation). The lower layer handles the control response (PID → Thrust Command → Physics Update).

2.4.3 Inertial Measurement Unit (IMU)

The vessel is equipped with a virtual 6-axis MEMS IMU, providing high-frequency updates on angular rates and linear accelerations. To facilitate robust state estimation, the simulator models a "Strapdown" inertial system attached to the vessel's Center of Gravity (CG).

Publisher Node (Virtualization)

The physics engine synthesizes IMU telemetry by transforming the calculated body-frame kinematics into proper acceleration and angular rates.

1. Angular Velocity (Gyroscope): The gyroscope output $\boldsymbol{\omega}_{gyro}$ correlates directly to the vessel's body-fixed rotational rates. While the simulator calculates full 3D orientation for visualization, the control logic focuses on the planar Yaw Rate (r), yielding the simplified output vector:

$$\boldsymbol{\omega}_{gyro} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \approx \begin{bmatrix} 0 \\ 0 \\ r \end{bmatrix} \tag{2.17}$$

2. Linear Acceleration (Accelerometer): The accelerometer measures "Proper Acceleration" ($\mathbf{f}^b$), which is the vector sum of kinematic acceleration, Coriolis/Centripetal terms, and the reaction to gravity. The general strapdown equation is:

$$\mathbf{f}^b = \dot{\mathbf{v}}^b + (\boldsymbol{\omega}^b \times \mathbf{v}^b) - \mathbf{R}_n^b \mathbf{g}^n \tag{2.18}$$

In the specific implementation, system assumes small roll/pitch angles and planar motion ($v \approx 0, w \approx 0$), this simplifies to the calculation used in the physics kernel:

$$\mathbf{a}_{sim} = \begin{bmatrix} \dot{u} \\ u \cdot r \\ g \end{bmatrix} \tag{2.19}$$

Note: The term $u \cdot r$ represents the centripetal acceleration experienced during a turn. Including this term is used for preventing "drift" in state estimators that rely on inertial integration, as it accounts for the rotating reference frame of the turning vessel.

Receiver Node

The control system ingests the raw IMU message to derive higher-order derivative states required for advanced model-based control.

Angular Acceleration Derivation: Standard IMUs provide angular velocity $\boldsymbol{\omega}$, but not angular acceleration. To anticipate inertial lags in the vessel's response (specifically the T parameter in Nomoto models), the system estimates the angular acceleration vector $\boldsymbol{\alpha}$ using a First-Order Backwards Difference method:

$$\boldsymbol{\alpha}[k] = \frac{\boldsymbol{\omega}_{meas}[k] - \boldsymbol{\omega}_{meas}[k-1]}{\Delta t} \tag{2.20}$$

This derivative term quantifies the rotational energy change, serving as a input for the "Update Physics" observer which validates the dynamic Nomoto parameters in real-time.

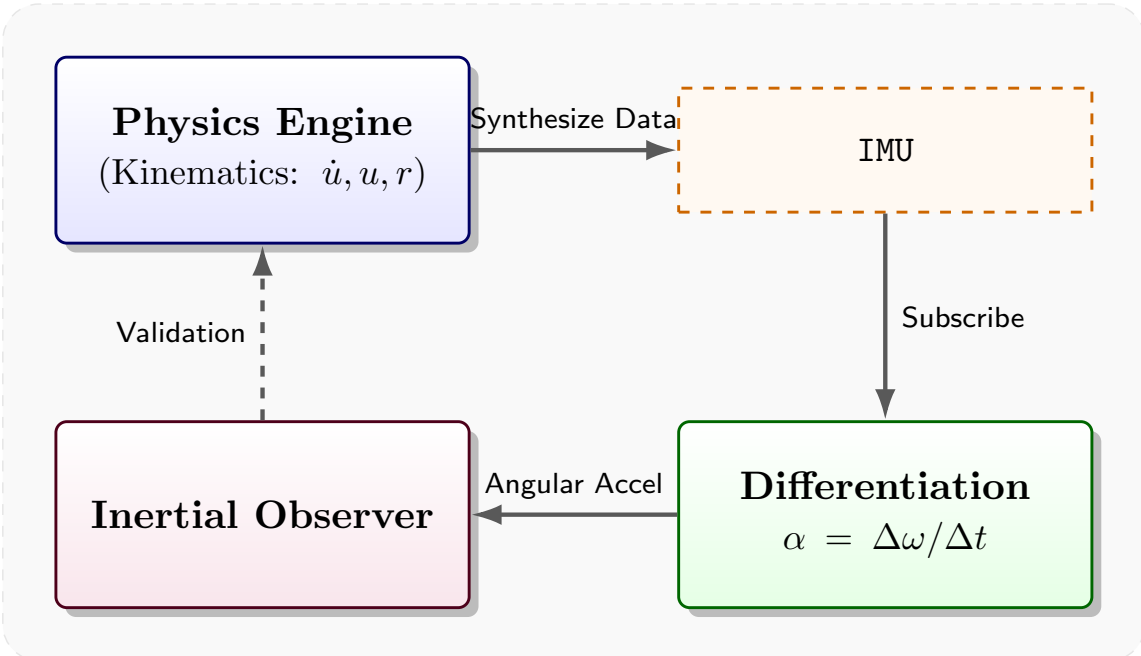

Figure 2.4: IMU Signal Topology: The Physics Engine computes proper acceleration. The Control Node differentiates gyroscope data to recover angular acceleration (α) for the observer.

2.4.4 Rudder Actuation Feedback

The steering system utilizes a virtual absolute encoder to provide closed-loop feedback for the heading controller, ensuring that the simulated physics engine and the control logic remain synchronized.

Data Structure

The actuator state is broadcast via the standard ROS 2 `joint states` topic. The message contains the raw angular displacement of the rudder stock δ, encoded in radians to adhere to SI standards.

Signal Processing

Upon receipt, the raw encoder signal undergoes normalization and conditioning before being utilized by the hydrodynamic solver.

1. Unit Conversion: To facilitate operator interpretation and consistent parameter scheduling, the radian input is converted to degrees:

$$\delta_{deg} = \delta_{rad} \cdot \left(\frac{180}{\pi}\right) \tag{2.21}$$

2. Limit Clamping (Mechanical Stops): To simulate physical steering constraints, the control logic saturates the input command to the vessel's effective rudder range. This prevents integral windup in the PID controller and ensures the Nomoto model operates within valid linearization bounds:

$$\delta_{eff} = \text{sat}(\delta_{raw}, \delta_{lim}) = \begin{cases} \delta_{lim} & \text{if } \delta_{raw} > \delta_{lim} \\ -\delta_{lim} & \text{if } \delta_{raw} < -\delta_{lim} \\ \delta_{raw} & \text{otherwise} \end{cases} \tag{2.22}$$

This effective angle δ_{eff} serves as the direct input to the variable Nomoto gain $K(\delta)$ and time constant $T(\delta)$ scheduling functions described in Section 2.3.

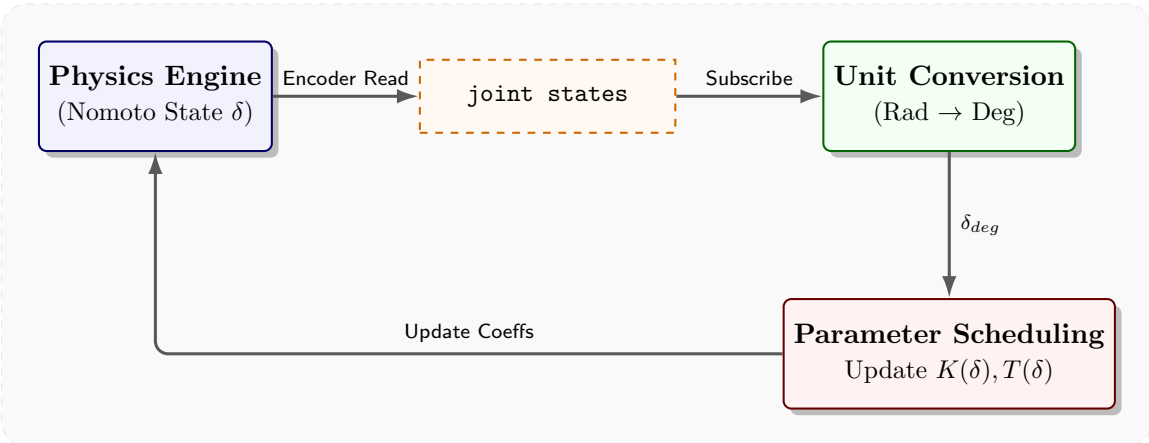

Figure 2.5: Rudder Feedback Topology: The closed-loop cycle where actuator state determines the hydrodynamic coefficients (K, T) for the next physics step.

2.4.5 Data Synchronization and Fusion

The ASV Control Center functions within a distributed ROS 2 architecture where sensor telemetry and actuator feedback arrive asynchronously. To maintain temporal coherence between the physics engine and the control logic, the system abstracts the asynchronous message passing into a Synchronous Sampled-Data System.

Deterministic Event Loop

Synchronization is enforced via a centralized Application Loop driven by a high-precision timer, rather than relying on non-deterministic ROS callbacks. Operating at a fixed frequency of $f_s = 20$Hz, this architecture ensures that simulation time advances linearly with wall-clock time, effectively decoupling the physics solver from network jitter.

The Control Cycle

In each discrete time-step k, the system executes a strict sequence of atomic operations to close the feedback loop:

1. **Data Ingestion (Spin):** The system explicitly triggers the ROS executor (spin once) to digest all pending messages in the subscriber queue.
2. **Zero-Order Hold (ZOH) Fusion:** Since sensors publish at different rates, the state estimator applies a Zero-Order Hold strategy. The internal state vector $\mathbf{x}[k]$ is updated using the *most recent* buffered values from the GNSS and IMU topics. If a sensor packet is delayed, the system holds the value from $\mathbf{x}[k-1]$, ensuring the solver never blocks waiting for data.
3. **Control Law Update:** The PID algorithms compute the required control efforts (Thrust u_{cmd} and Rudder δ_{cmd}) based on the synchronized state snapshot $\mathbf{x}[k]$.
4. **Physics Integration:** Finally, the Nomoto solver advances the vessel's kinematic state for the interval $[t, t + \Delta t]$ using the computed control inputs, guaranteeing a deterministic time-step ($dt = 0.05s$) for the numerical integration.

2.5 Control System Design

The ASV control architecture is designed to manage two primary DoF: Surge Speed and Heading. The system operates in three distinct selectable control modes allowing for flexible testing of both open-loop actuation and closed-loop autonomy.

2.5.1 Control Architecture Overview

The system implements a cascaded decision logic where the active source of control signals is determined by the selected mode.

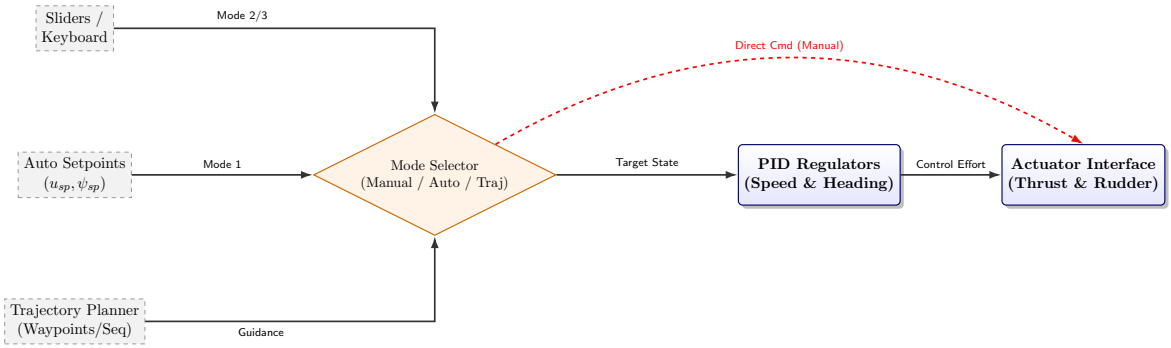

Figure 2.6: Control System Architecture: The central switching logic routes either raw operator commands (Manual) or PID-computed efforts (Auto/Trajectory) to the vessel actuators.

2.5.2 Manual Control Modes

Manual operation allows operators to inject open-loop signals directly into the physics engine, bypassing the PID regulators.

Slider Control (Direct)

In "Slider Mode," the GUI widgets map directly to the actuator limits. The command vector $\mathbf{u}_{cmd}$ is derived as:

$$u_{cmd}(t) = \frac{\text{Slider}_u}{100} \cdot V_{max} \tag{2.23}$$

$$\delta_{cmd}(t) = \text{Slider}_r \quad [\text{deg}] \tag{2.24}$$

where $V_{max} = 5.0$ m/s. This provides a "Step Input" capability for analyzing transient response.

Keyboard Control (Teleoperation)

In "Arrow Mode," the system implements a dynamic accumulator with auto-centering decay, mimicking a physical joystick's return-to-center behavior.

- **Acceleration:** When a key is held, the command increments by $\alpha = 0.2$.
- **Decay:** When released, the command decays towards zero by $\beta = 0.1$.

The discrete update law implemented in the event loop is:

$$u[k] = \begin{cases} u[k-1] + \alpha & \text{if Key}_{\uparrow} \\ u[k-1] - \alpha & \text{if Key}_{\downarrow} \\ u[k-1] - \text{sgn}(u[k-1]) \cdot \beta & \text{else (Decay)} \end{cases} \tag{2.25}$$

A similar logic applies to the rudder with an increment of 1.5° and a strong centering decay (2.0°) to simulate hydrodynamic realignment.

2.5.3 Automatic Control (PID)

In "Auto Mode," the system activates two independent PID controllers to regulate Speed and Heading. The controllers operate on the error signal calculated at 20Hz.

Speed Controller

The surge velocity is regulated by a standard PID acting on the error $e_u[k] = u_{sp} - u_{meas}[k]$.

$$u_{out} = K_p e_u + K_i \sum e_u \Delta t + K_d \frac{e_u - e_{u,prev}}{\Delta t} \tag{2.26}$$

To prevent integrator windup during saturation (e.g., full throttle), the integral term I is clamped. The final output is saturated to the vessel's speed limits.

Heading Controller

Heading control requires special handling of the angular domain to avoid discontinuity at $\pm 180°$. The error is normalized to the shortest path:

$$e_\psi = \text{atan2}(\sin(\psi_{sp} - \psi_{meas}), \cos(\psi_{sp} - \psi_{meas})) \tag{2.27}$$

The PID output is inverted (negative feedback) to drive the rudder angle δ:

$$\delta_{cmd} = -1.0 \cdot \left(K_p e_\psi + K_d \frac{e_\psi - e_{\psi,prev}}{\Delta t} \right) \tag{2.28}$$

Note: The derivative term is computed on the error, which effectively provides rate damping ($K_d \dot{e} \approx -K_d r$) assuming a constant setpoint.

2.6 Trajectory and Mission Planning

The system includes a high-level guidance layer capable of executing waypoint-based paths and pre-programmed rudder sequences.

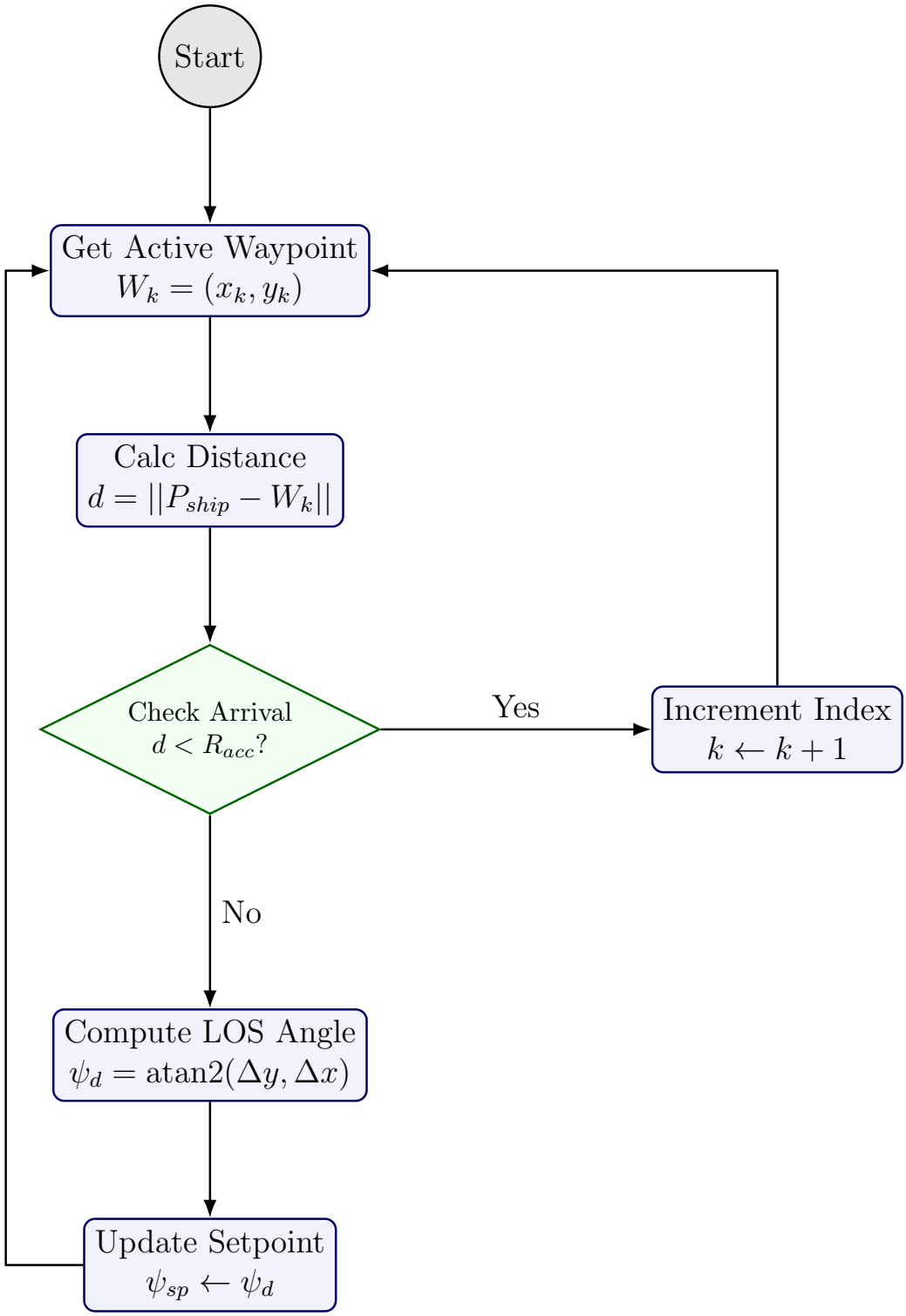

Figure 2.7: The Line-of-Sight (LOS) guidance loop

2.6.1 Path Planner

Autonomous navigation utilizes a Line-of-Sight (LOS) guidance law. The planner decomposes a global path defined by a list of coordinate tuples $W = \{w_0, w_1, ..., w_n\}$.

Guidance Law

For the active target waypoint $w_k = (x_k, y_k)$, the desired heading ψ_d is computed geometrically relative to the vessel's current position (x, y):

$$\psi_d(t) = \text{atan2}(y_k - y(t), x_k - x(t)) \tag{2.29}$$

This desired heading is fed directly into the Heading PID controller ($\psi_{sp} = \psi_d$).

Waypoint Switching Logic

To ensure smooth path transitions, the sequencer monitors the Euclidean distance $d(t)$ to the target. A "Sphere of Acceptance" criterion triggers the switching event:

$$\text{Trigger}(k \to k+1) \iff \sqrt{(x_k - x)^2 + (y_k - y)^2} < R_{acc} \tag{2.30}$$

In the current implementation, the acceptance radius is set to $R_{acc} = 1.0$ meter.

2.6.2 Rudder Sequence Scheduler

For validating hydrodynamic parameters, the system can execute a deterministic "Rudder Schedule" (e.g., Zig-Zag maneuvers). This mode bypasses the heading PID and injects timed open-loop commands.

The sequence is defined as a queue of tuples $Q = \{(\delta_0, t_0), (\delta_1, t_1), ...\}$. A dedicated timer executes the queue:

$$\delta_{cmd}(t) = \delta_i \quad \text{for} \quad t_{start,i} \leq t < t_{start,i} + t_i \tag{2.31}$$

2.7 Software Initialization

Upon launching the application, the user is greeted by the Splash Screen (Figure 2.8). This landing page performs background checks on the ROS 2 middleware connectivity.

Figure 2.8: Application Entry Point: The ASV Sim Splash Screen allows the user to initialize the simulation kernel or access documentation before entering the main control loop.

Selecting "Run Simulation" initializes the backend services and opens the main operational interface. The Documentation button provides access to the technical documentation. If Skip Guide is not enabled, the basic helper guide is automatically launched. The Exit buttons terminate the application.

2.8 Main Dashboard Overview

The core interface is designed for situational awareness. As shown in Figure 2.9, the "Current Status" panel at the top-left provides a numerical readout of the vessel's state vector.

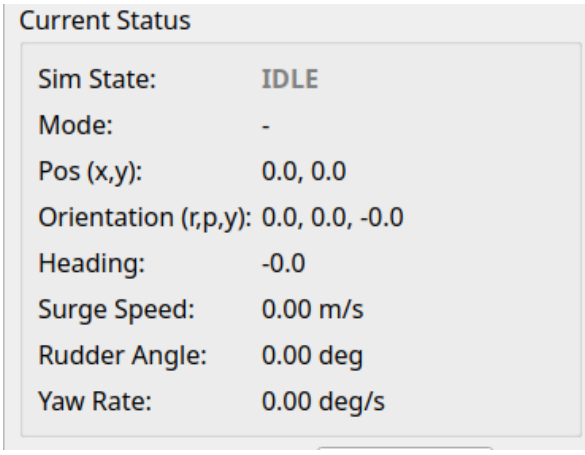

Figure 2.9: The Current Status Panel

Status Indicators:

- **Sim State:** Indicates if the physics engine is integrating (‘RUNNING‘), frozen (‘PAUSED‘) or Resets all it states to initial conditions and waiting for re-initiation (‘IDLE‘).
- **Mode:** Showing the active control mode
- **Pos (x,y):** Global position in meters relative to the start origin.
- **Orientation (r,p,y):** Orientations (roll, pitch, yaw) in degrees
- **Heading:** The vessel’s current Yaw angle (ψ) in degrees.
- **Surge Speed:** Forward velocity (u) in m/s.
- **Rudder Angle:** Current in command rudder angle in degrees
- **Yaw Rate:** Measured instantaneous yaw ratw value

2.9 Simulation Lifecycle Management

Located at the top-left of the Control Sidebar (Figure 2.10), the Simulation Control group manages the execution state of the physics kernel.

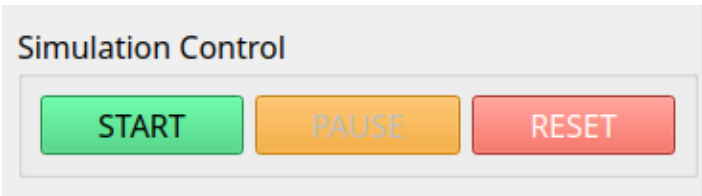

Figure 2.10: Global Simulation Controls

The interface provides three distinct states:

- **START :** Engages the application timer. This triggers the loop callback at 20Hz, enabling real-time physics integration, sensor publishing, and controller execution.
- **PAUSE :** Freezes the simulation time t. The vessel's kinematic state vector $\mathbf{x}(t)$ is preserved in memory, allowing the operator to inspect instantaneous values without the system drifting.
- **RESET :** Performs a "Hard Reset". This action:
  1. Zeros the state vector (Position $x = 0, y = 0$, Heading $\psi = 0$).
  2. Flushes the PID integrator buffers (preventing "integral windup" from previous runs).
  3. Resets the simulation clock to $t = 0.0s$.

2.10 Control Modes

The "Controls" tab allows the operator to select the active input source. The system supports three distinct modes of operation.

2.10.1 Manual Control (Sliders & Keyboard)

For open-loop testing and step-response analysis, the **Manual (Sliders)** mode is used. This connects the GUI widgets directly to the simulated actuators.

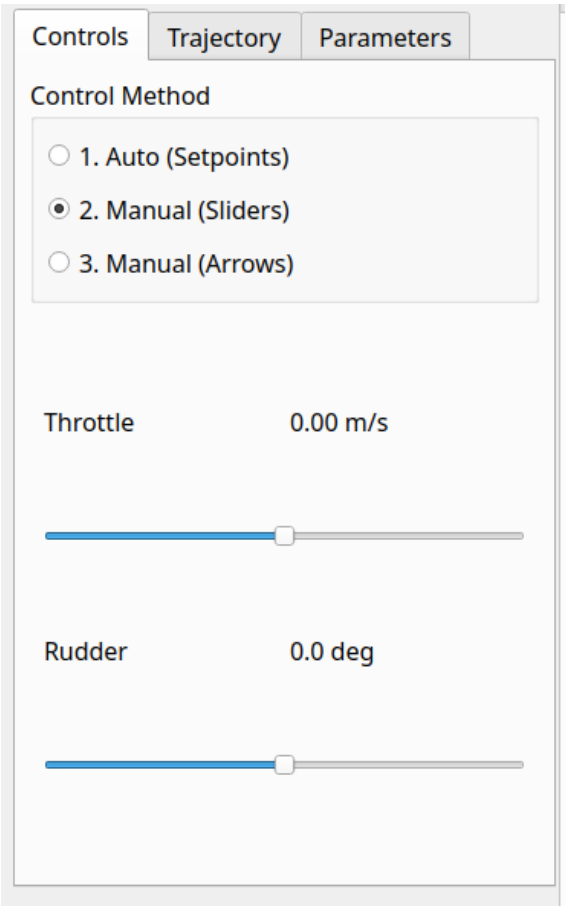

Figure 2.11: Manual Control (Slider).

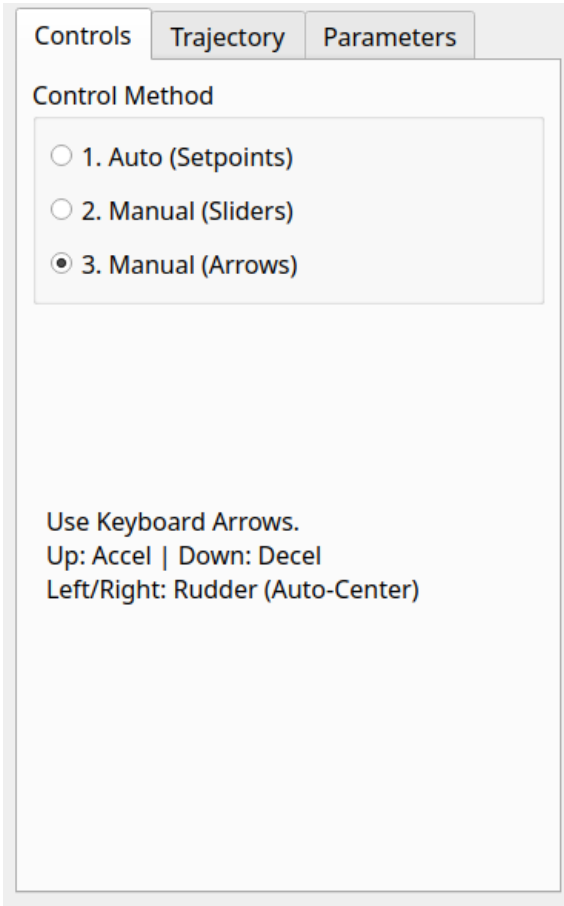

Figure 2.12: Manual Control (Arrows).

- **Throttle Slider:** Sets the target surge speed (u_{cmd}) between -5.0 and $+5.0$ m/s.
- **Rudder Slider:** Sets the rudder angle (δ_{cmd}) within the mechanical limits of $\pm 17°$.
- **Keyboard Shortcuts:** In "Arrow Mode," the Up/Down keys increment throttle, while Left/Right keys actuate the rudder with an auto-centering decay to simulate a joystick.

2.11 Trajectory and Mission Planning

The "Trajectory" tab switches the system to autonomous guidance. This interface allows users to design paths or schedule deterministic maneuver sequences.

2.11.1 Waypoint Navigation

The Waypoint Planner (Figure 2.13) enables Point-to-Point navigation using the Line-of-Sight (LOS) guidance law.

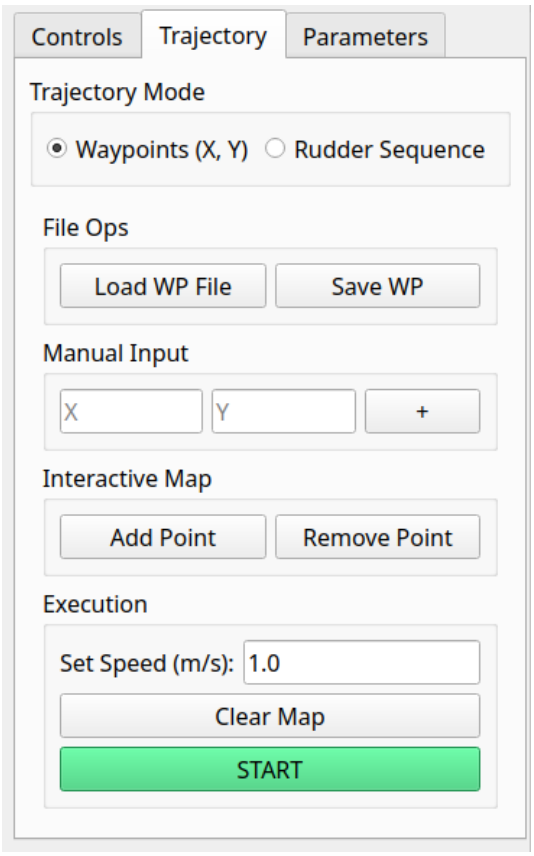

Figure 2.13: Trajectory Planner Panel.

Operational Features:

- **Interactive Map:** When "Add Point" is toggled, clicking anywhere on the 2D grid (Right Panel) appends a coordinate to the mission queue. In order to remove one-by-one "Remove Point" options must be used. Clear map option clear all the points at once.
- **File Ops:** Standard '.txt' files containing 'x,y' coordinates can be loaded via browsing with 'Load WP File' button for repeatable mission profiles v. If the edited map is wanted to preserve for later use 'Save WP' will open a folder browser for user to select where to save waypoints file (same format as load wp file).
- **Execution:** Setting the "Set Speed" and clicking **START** engages the Autopilot. The vessel will autonomously navigate the waypoints in sequence.

2.11.2 Rudder Sequence Scheduler

For hydrodynamic parameter identification (e.g., Zig-Zag tests), the Rudder Sequence mode (Figure 2.14) is used. This bypasses the heading controller to execute a timed queue of open-loop commands.

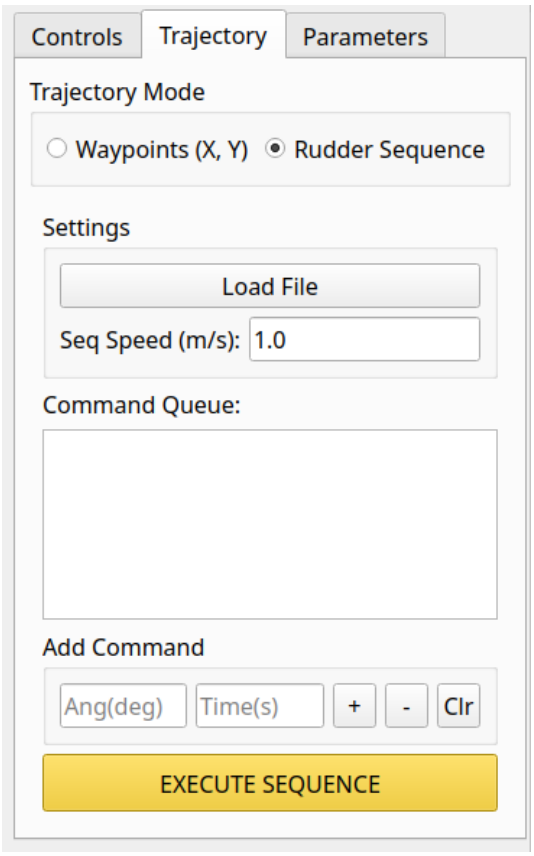

Figure 2.14: Rudder Sequence Scheduler Panel

Operational Features:

- **Global Sequence Speed:** Before defining the turns, the user sets a constant surge velocity (e.g., 2.0 m/s) in the "Seq Speed" field. This ensures the vessel maintains sufficient kinetic energy throughout the maneuver.
- **Command Queue Construction:** The user can either load pre-determined rudder schedule from 'Load File' or can build the mission profile iteratively. By entering a specific Rudder Angle (degrees) and Duration (seconds) and clicking the '+' button, the tuple is added to the "Command Queue" list widget (e.g., "Rudder: 15° for 5.0s").
- **Queue Management:** The interface provides tools for error correction; the **'-'** button removes the selected command, while the **'Clr'** button purges the entire schedule.
- **Real-Time Execution:** Clicking **EXECUTE SEQUENCE** (Yellow Button) engages the mission timer. The system automatically highlights the currently active command in the list view as it progresses, providing the operator with immediate visual feedback on the maneuver's stage.

2.12 Parameter Tuning

The "Parameters" tab (Figure 2.15) provides a direct interface to the physics engine's internal coefficients.

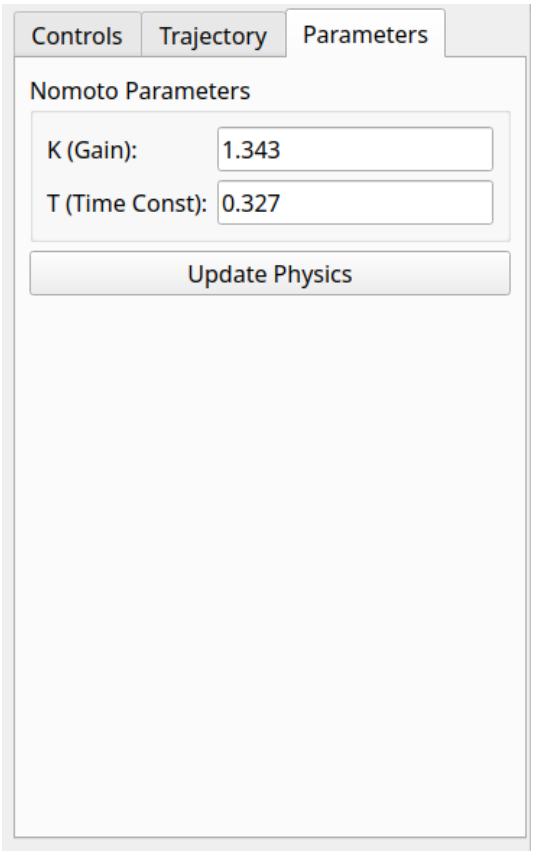

Figure 2.15: Real-Time Parameter Injection Panel

This feature enables the application to be used with different types of marine vessels, rather than being limited to a single vessel type.

2.13 Real-Time Visualization

The Right Panel is dedicated to high-fidelity data visualization, essential for analyzing system performance.

2.13.1 Time-Series Plots

Three stacked graphs display the history of the last 25 seconds (500 samples):

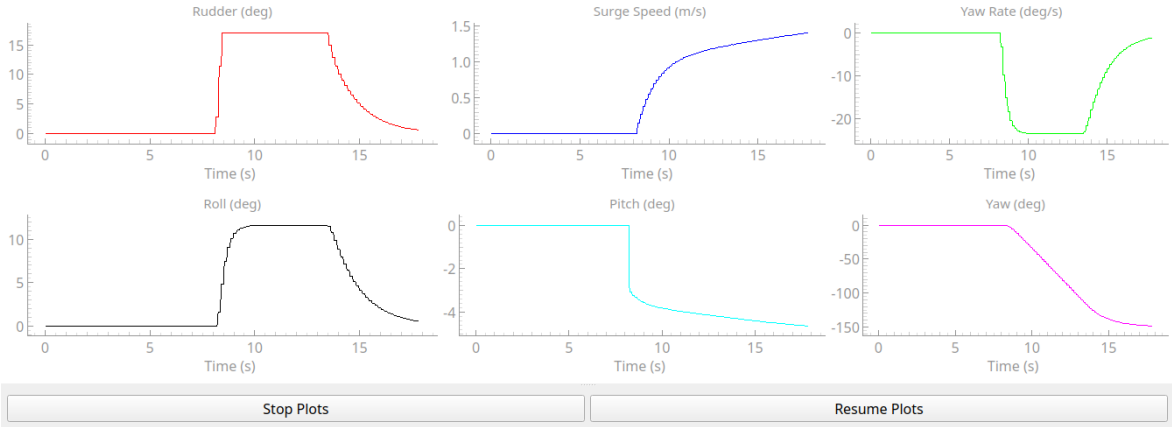

Figure 2.16: Time-Series Plots

2.13.2 2D Trajectory Map

The bottom grid creates a spatial "God's Eye View" of the simulation.

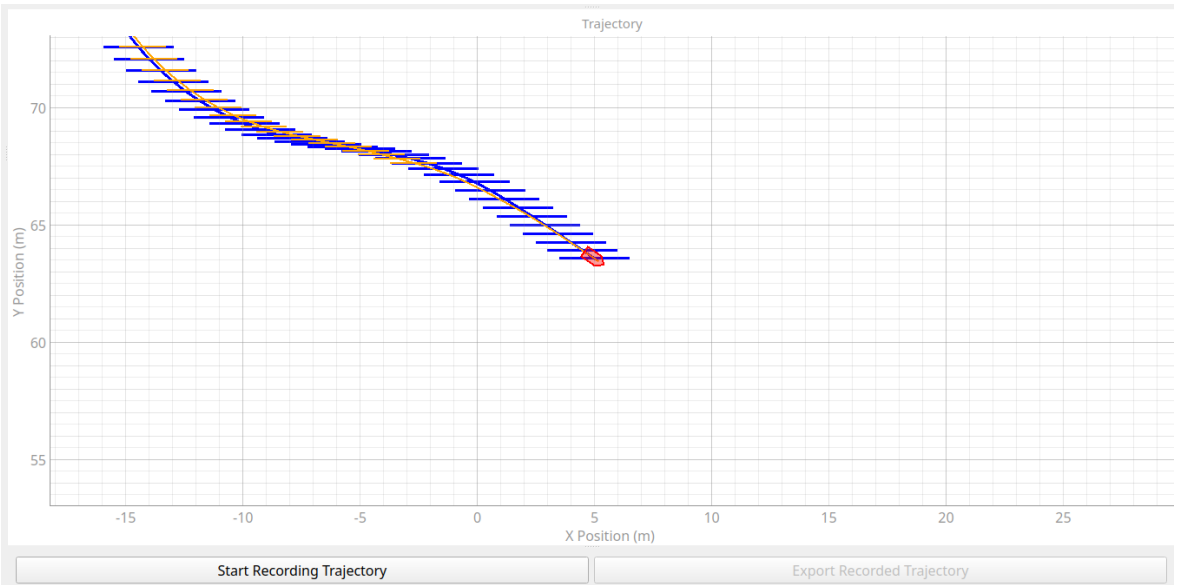

Figure 2.17: Trajectory Panel

- **Blue Trace:** The vessel's actual path.
- **Orange Trace:** The "Nomoto Prediction" path (ideal model).
- **Red Dashed Line:** The active Waypoint path (if in Trajectory mode).
- **Vessel Icon:** A red polygon representing the ship's current position and orientation (ψ).

Beneath the trajectory panel, two buttons are provided for video trajectory recording. Selecting Start Recording Trajectory opens a folder selection dialog, allowing the

user to choose the destination directory. From that point onward, video recording begins and is continuously written to the selected location until the user selects Export Recording Trajectory, which finalizes the recording.

2.14 Data Logging and Export

For post-mission analysis, the ASV Control Suite includes a built-in telemetry recorder located at the bottom of the sidebar (Figure 2.18).

Figure 2.18: Data Recording Interface: Tools for managing the visual workspace and exporting high-fidelity simulation data for external analysis.

2.14.1 Visual Trace Management

During long-duration simulations, the 2D map can become cluttered with overlapping path history.

- **Release Lock:** By checking "Release Lock (Zoom/Pan)," the user decouples the camera from the vessel, allowing free exploration of the map using the mouse wheel and drag gestures.
- **Clear Trajectory Traces:** Clicking this button wipes the visual history (Blue/Orange lines) from the plotting widget *without* resetting the physics engine. This is useful for clearing the screen between repetitive maneuvers.

2.14.2 Telemetry Export

The system automatically buffers all sensor data and internal state variables into a RAM history queue while the simulation is `RUNNING`.

Export Workflow:

1. **Suspend Simulation:** It is recommended to transition the simulation state to `PAUSED` before exporting. This freezes the data buffer, ensuring that the exported dataset represents a complete and consistent time-slice without write conflicts.
2. **Initiate Export:** Clicking the **"Export Data"** button invokes the operating system's native file dialog, allowing the user to define the destination path.

3. **Format Selection:** The system supports saving data in either `.csv` (Comma Separated Values) for universal compatibility or `.xlsx` (Microsoft Excel) for immediate spreadsheet analysis.

The exported telemetry file is structured with high-precision floating-point values, organized into the following column groups:

- **Temporal Data:** Simulation elapsed time (t_{sim}) and the corresponding Real-world UTC timestamp for log synchronization.
- **Navigational State:** Global Cartesian coordinates (x, y) and the normalized Heading angle (ψ).
- **Vessel Dynamics:** Surge Velocity (u), Yaw Rate (r), and estimated Euler attitudes for Roll (ϕ) and Pitch (θ).
- **Actuator Feedback:** The instantaneous Rudder Angle (δ) and active Throttle commands.